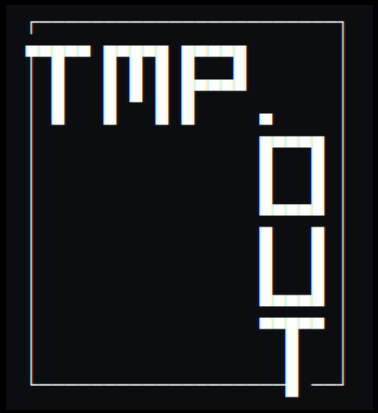
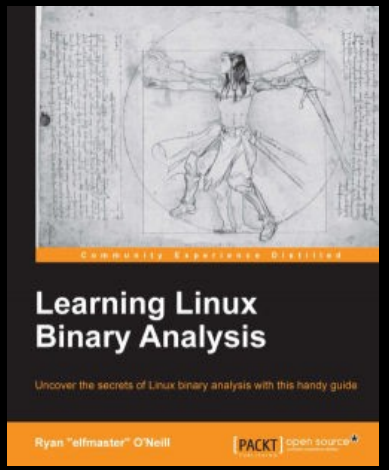
Revolutionizing ELF binary patching with Shiva

By ElfMaster

Introduce myself...

Ryan “ElfMaster” O’Neill
Author: Phrack, POC||GTFO, Vxheaven, tmp.0ut, bitlackeys.org





Founder of Arcana Research
Founder of Arcana Technologies

- https://arcana-research.io
  - Research & development technologies
  - Shiva, AMP phase-3
- https://arcana-technologies.io
  - Linux threat detection
  - Detect hidden threats within ELF artifacts

DARPA AMP PROGRAM: PHASE-2

- Dr. Sergey Bratus
- Assured Micro Patching phase-2
- Advancing the state of the art:
  - ELF reverse engineering tools
  - Binary patching technologies
  - Patch verification
  - Various Arch's: ARM32/ARM64, RISCV, SPARC, PowerPC
- Now in transition phase-3

What is Shiva?

- Next-level binary patching solution
  - Custom ELF interpreter for Linux AArch64
  - An ELF loader, linker, and transformer
  - Re-program native ELF software
- Built on 17 years of ELF expertise and research

What makes it special?

- Innovations in ELF linking concepts
- Advanced ELF program transformation
- Patching mechanics are driven by natural C code
- Compatible with existing ELF ABI toolchain
- Allows developers to write nuanced patches with ease

How can Shiva help you?

- Companies and organizations:
  - Re-program legacy software to fix bugs in mission critical systems
  - Use Shiva as a modular patch system to update software
- Other use cases
  - Advanced instrumentation engine
  - Black-box fuzzing harnesses
  - Software tracing and profiling instruments

Origins of Shiva

- An itch to write a custom ELF interpreter
- Led to “Interpreter chaining” and a modular ELF virus loader
- tmp.0ut paper: “Preloading the linker for fun and profit” https://tmpout.sh/2/6.html

Origins of Shiva

- The original runtime linker for tmp.0ut morphed into https://github.com/elfmaster/shiva
  - An early version of Shiva for x86_64
  - Load ELF microprograms for sophisticated instrumentation: security features, tracing engines, in-process debugging, etc.
  - A reverse engineers dream toy

Origins of Shiva

- Shiva Alpha v0.11 (Present model)
- AArch64 Linux
- Forked off from the original prototype
- Heavily modified and tailored to be a binary patching system
- Designed for phase-2 of DARPA AMP (Assured micro patching)

Rewind 25 years...

- Who was originally trying to patch ELF binaries?
- And Why? ...

Rewind 25 years…
the era of early UNIX virus research

- Silvio Cesare published “UNIX and ELF parasites” 1998
- A plethora of ELF infection research
- The esoterica of ELF had begun…
- I was enamored by this unique and enigmatic research

ELF Virus technology & instrumentation

- Complex ELF re-structuring techniques
- Code and data storage injection
- Code re-linking: *plt hooks, inline hooks, relocation poisoning, etc.*

- PTRACE injection & process control
  - Hot patching the ELF runtime
  - Runtime instrumentation

- An exploration by researchers to understand and abuse various ELF components

Early influential ELF research from the Underground scene

- The userland exec implementation by Grugq
  - Anti-forensics research
  - ELF binary protection stubs use ul_exec
- ERESI & Embedded ELF debugging. Mayhem.
  - Advanced re-linking and code injection
  - In-process injection of debugger
- Silvios “/dev/kmem” ET_REL injection
  - Earliest example of a custom object-file linker/injector
  - A version of insmod that injects ET_REL objects into kernel via /dev/kmem

Academic ELF research

- **Katana**: A dwarf-aware hotpatching framework for ELF. “Sergey Bratus & James Oakley”
- **PEBIL** (Efficient static binary instrumentation for Linux) “Michael A. Laurenzano”.
- **Weird machines in ELF** (rtld meta-data). “Rebecca Shapiro, Sergey Bratus, Sean W. Smith.”

ELF binary instrumentation: A double edged sword

- ELF instrumentation can learn much from the world of ELF backdoors, viruses, and rootkits
- ELF binary patching is an old topic, as old as UNIX viruses... or is it even older?

1995: ELF (Executable linking format) Solutions in plain sight...

- ELF was inherently designed to patch code:
  - “/bin/ld” links relocatable code (Why not re-link too?)
  - ld-linux.so is a JIT micropatching engine
  - ELF relocations describe patching operations
  - Why not extrapolate on that concept?
  - Linking + Transformation = Patching
- Custom ELF interpreters have promise...

The power of Hobbyist ELF interpreters

- Invoked by the kernel
  - “/lib/shiva” embedded in PT_INTERP
  - Sets up the finer details of the process image
  - The Interpreter has instrumentation precedence

The power of Hobbyist ELF interpreters

- Fast In-process instrumentation
- Faster than PTRACE hot-patching
- Can transform, link, and instrument the program
- ELF relocatable objects (ET_REL) contain granular relocation meta-data

A hybrid of “/bin/ld” and “ld-linux.so”

- Shiva draws wisdom from both linkers
- Flexible and modular like ld-linux.so
- With the nuance of ld; relocatable objects
- Hybridized linking technology

Definitions of ELF linking components

- “/bin/ld”
  - An ELF linker
- “/lib/ld-linux.so”:
  - An ELF linker and loader
- “/lib/shiva”:
  - An ELF linker, loader and transformer

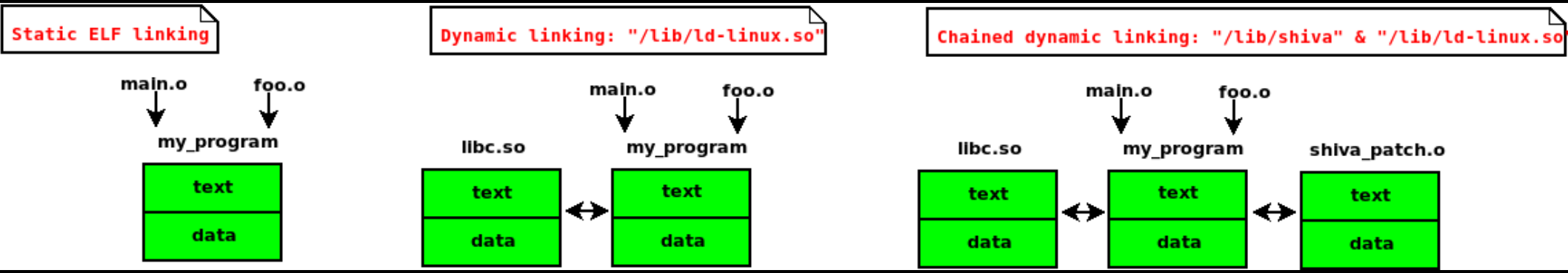
Shiva's Innovation in ELF linking concepts

- Revolutionizing the ELF runtime
  - Chained linking, aka "Interpreter chaining"
  - ELF Transforms
  - RTLD meta-data programming
  - The Shiva Pre-linker "/bin/shiva-ld"
  - Cross Relocations (Confluent linking operations)
  - Generates on-the-fly relocations for binaries

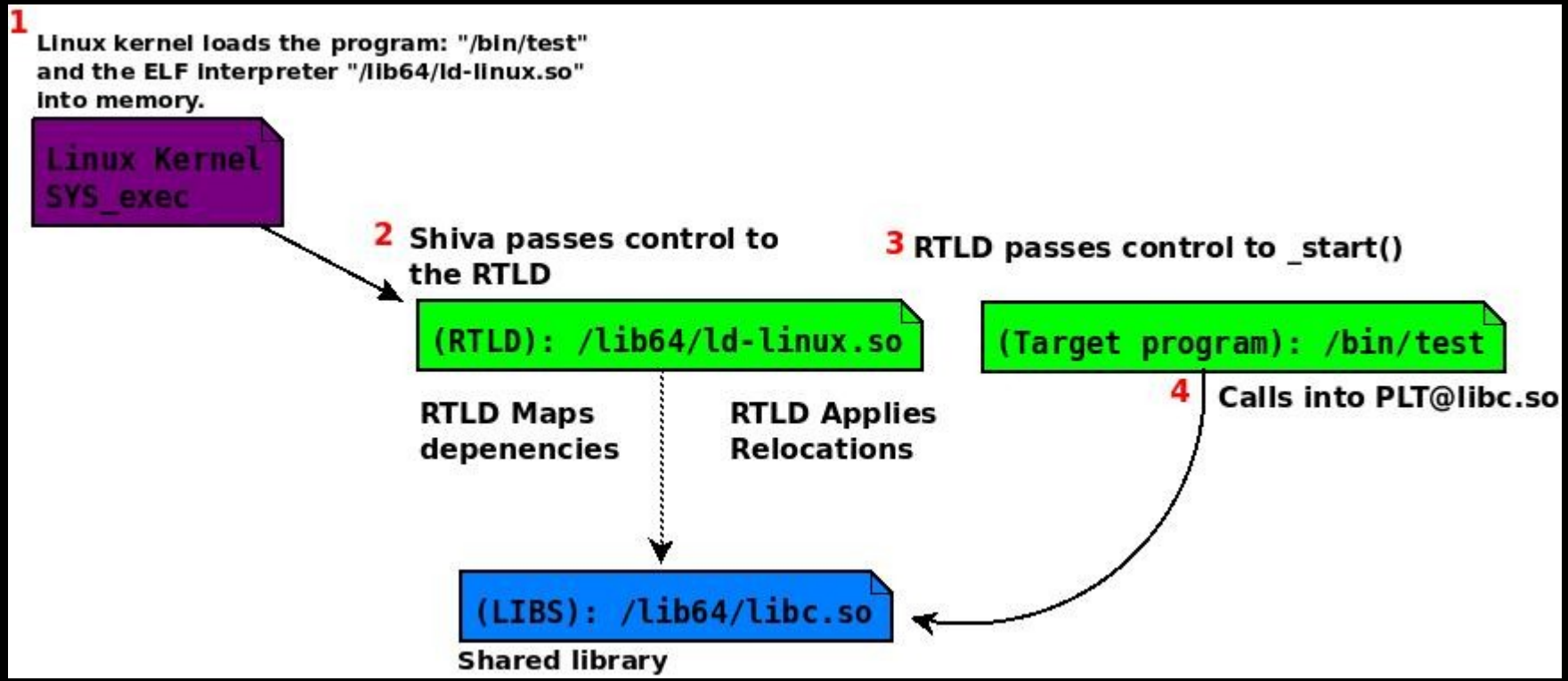
Shiva has libelfmaster under the hood

- Built on top of libelfmaster
  - Advanced ELF parsing library
  - Forensic reconstruction of stripped ELF binaries
  - Rebuilds section header table and symbol tables
  - Forensically reconstructs .symtab and .dynsym
  - https://github.com/elfmaster/libelfmaster

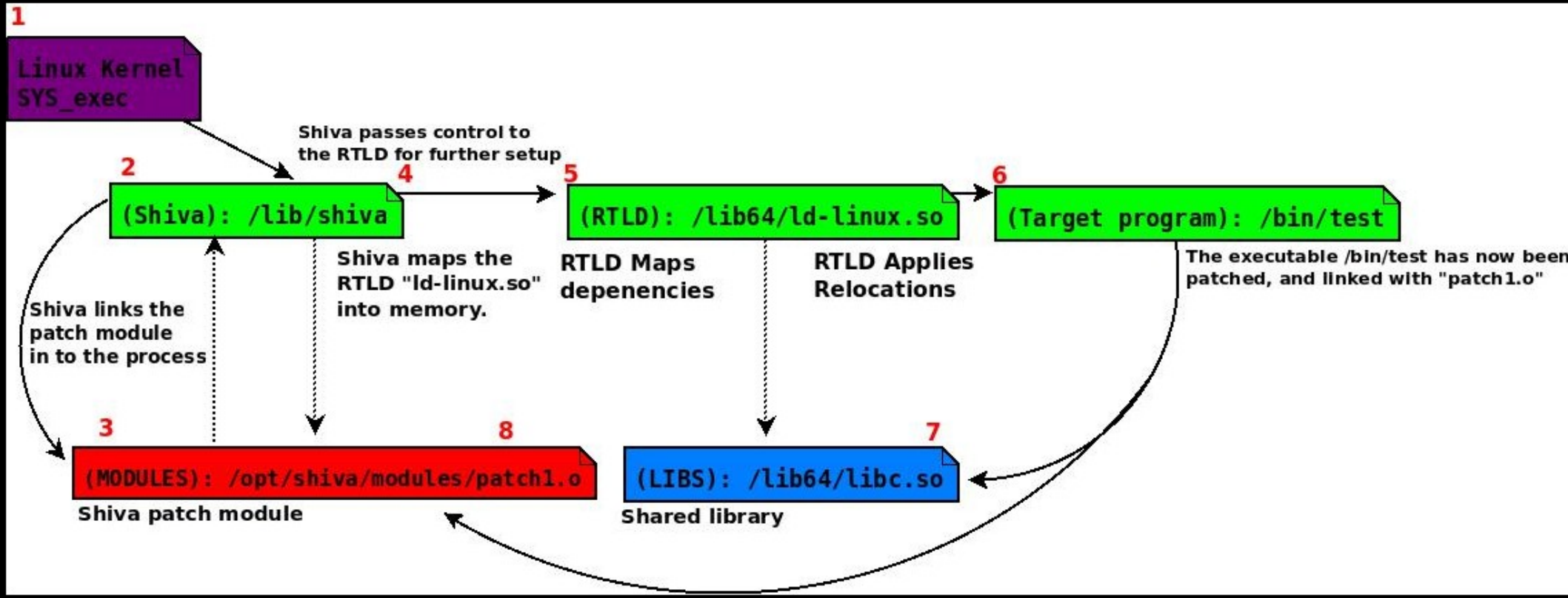
Shiva's ELF linking workflow



Standard ELF dynamic linking workflow



ELF “Linker chaining” diagram



ELF Patch objects: Relocatable code

- ELF ET_REL objects: i.e. patch.o
- Large code model: gcc -mcmodel=large
  - Patch modules have an internal .got.plt[] of their own
  - R_AARCH64_ABS64 .text encodings hold absolute addresses for xrefs
- Rich relocatable & symbolic meta-data
- Shiva builds a process image for the patch object.
- See source for: shiva_module_loader()

ELF Executable re-linking

- ELF executables (ET_DYN || ET_EXEC):
  - Basic RTLD relocations (.got, .got.plt, .init_array, etc.)
  - Lack relocation data describing how to re-link the .text
- Shiva generates relocations on the fly for the program
  - A CFG is generated by shiva-ld (Or shiva at runtime)
  - Modifies dynamic relocation records to re-link some variables and code
  - Shiva can transform and re-link entire functions

Shiva's ulexec functionality

- Shiva can be executed directly to load the binary it is patching:

```
$ SHIVA_MODULE_PATH=./patch.o
$ shiva <binary>
```

- Shiva loads the executable via a userland exec implementation
- Patches can be tested without any modification to target binary (i.e. without modifying PT_INTERP)

Shiva as an ELF interpreter

- Modify PT_INTERP
  - “/lib/ld-linux.so” → “/lib/shiva”
- Dynamic segment: New entries describing patch meta-data
- The Shiva-Prelinker “/bin/shiva-ld” installs this meta-data into the binary

Shiva interpreter path: “/lib/shiva”

```
Elf file type is DYN (Shared object file)
Entry point 0x6b0
There are 8 program headers, starting at offset 64

Program Headers:
  Type           Offset             VirtAddr           PhysAddr
                 FileSiz            MemSiz              Flags  Align
  PHDR           0x0000000000000040 0x0000000000000040 0x0000000000000040
                 0x00000000000001c0 0x00000000000001c0  R      0x8
  INTERP         0x0000000000000200 0x0000000000000200 0x0000000000000200
                 0x000000000000001b 0x000000000000001b  R      0x1
      [Requesting program interpreter: /lib/shiva]
  LOAD           0x0000000000000000 0x0000000000000000 0x0000000000000000
                 0x00000000000008e8 0x00000000000008e8  R E    0x10000
  LOAD           0x0000000000000d70 0x0000000000010d70 0x0000000000010d70
                 0x00000000000002a0 0x00000000000002a8  RW     0x10000
  DYNAMIC        0x0000000000003000 0x0000000000012000 0x0000000000012000
                 0x00000000000001d0 0x00000000000001d0  RW     0x8
  LOAD           0x0000000000003000 0x0000000000012000 0x0000000000012000
                 0x0000000000000260 0x0000000000000260  RWE    0x1000
  GNU_STACK      0x0000000000000000 0x0000000000000000 0x0000000000000000
                 0x0000000000000000 0x0000000000000000  RW     0x10
  GNU_RELRO      0x0000000000000d70 0x0000000000010d70 0x0000000000010d70
                 0x0000000000000290 0x0000000000000290  R      0x1
```

Updated dynamic segment

```
Dynamic section at offset 0x3000 contains 29 entries:
  Tag        Type                         Name/Value
 0x0000000000000001 (NEEDED)             Shared library: [libc.so.6]
 0x000000000000000c (INIT)               0x608
 0x000000000000000d (FINI)               0x8b4
 0x0000000000000019 (INIT_ARRAY)         0x10d70
 0x000000000000001b (INIT_ARRAYSZ)       8 (bytes)
 0x000000000000001a (FINI_ARRAY)         0x10d78
 0x000000000000001c (FINI_ARRAYSZ)       8 (bytes)
 0x000000006ffffef5 (GNU_HASH)           0x260
 0x0000000000000005 (STRTAB)             0x3a0
 0x0000000000000006 (SYMTAB)             0x280
 0x000000000000000a (STRSZ)              149 (bytes)
 0x000000000000000b (SYMENT)             24 (bytes)
 0x0000000000000015 (DEBUG)              0x0
 0x0000000000000003 (PLTGOT)             0x10f70
 0x0000000000000002 (PLTRELSZ)           168 (bytes)
 0x0000000000000014 (PLTREL)             RELA
 0x0000000000000017 (JMPREL)             0x560
 0x0000000000000007 (RELA)               0x470
 0x0000000000000008 (RELASZ)             240 (bytes)
 0x0000000000000009 (RELAENT)            24 (bytes)
 0x000000000000001e (FLAGS)              BIND_NOW
 0x000000006ffffffb (FLAGS_1)            Flags: NOW PIE
 0x000000006ffffffe (VERNEED)            0x450
 0x000000006fffffff (VERNEEDNUM)         1
 0x000000006ffffff0 (VERSYM)             0x436
 0x0000000060000018 (SHIVA_NEEDED)       Patch object: [patch.o]
 0x0000000060000017 (SHIVA_SEARCH)       Search path: [/opt/shiva/modules]
 0x0000000060000019 (SHIVA_ORIG_INTERP)  Original interp: [/lib/ld-linux-aarch64.so.1]
 0x0000000000000000 (NULL)               0x0
```

Rewrite, re-link, and reprogram!

- **Rewrite:** Write your patch in C and compile into an ELF object file.
- **Re-link:** At runtime Shiva will load,

ELF Symbol interposition

- Re-write symbolic code and data on the fly with ease!
  - Think LD_PRELOAD
  - Replace any function in the executable
  - Replace all global data: .rodata, .data, .bss
- No TLS support yet

Symbol interposing example:
Re-write .rodata string in helloworld program

```
elfmaster@esoteric-aarch64:~/hello_world$ ./hello
Hello World!
elfmaster@esoteric-aarch64:~/hello_world$ readelf -s hello | grep hello_string
    68: 0000000000000830    13 OBJECT  GLOBAL DEFAULT   15 hello_string
elfmaster@esoteric-aarch64:~/hello_world$ readelf -S hello | grep rodata
  [15] .rodata           PROGBITS         0000000000000828  00000828
elfmaster@esoteric-aarch64:~/hello_world$
```

Examine helloworld patch

```
elfmaster@esoteric-aarch64:~/hello_world$ cat patch.c

const char hello_string[] = "Hello DEFCON 31!";

elfmaster@esoteric-aarch64:~/hello_world$ make patch
gcc -mcmodel=large -fno-pic -I ../ -fno-stack-protector -c patch.c
elfmaster@esoteric-aarch64:~/hello_world$ file patch.o
patch.o: ELF 64-bit LSB relocatable, ARM aarch64, version 1 (SYSV), not stripped
elfmaster@esoteric-aarch64:~/hello_world$
```

Shiva installs patch at runtime only

```
elfmaster@esoteric-aarch64:~/hello_world$ ./hello
Hello World!
elfmaster@esoteric-aarch64:~/hello_world$ export SHIVA_MODULE_PATH=./patch.o
elfmaster@esoteric-aarch64:~/hello_world$ shiva ./hello
Hello DEFCON 31!
elfmaster@esoteric-aarch64:~/hello_world$ ./hello
Hello World!
elfmaster@esoteric-aarch64:~/hello_world$ shiva ./hello
Hello DEFCON 31!
elfmaster@esoteric-aarch64:~/hello_world$
```

Shiva Prelinker: Pre-link the patch to helloworld binary

```
elfmaster@esoteric-aarch64:~/hello_world$ shiva-ld -e hello -p patch.o -i /lib/shiva -s /opt/shiva/modules -o hello
[+] Input executable: hello
[+] Input search path for patch: /opt/shiva/modules
[+] Basename of patch: patch.o
[+] Output executable: hello
Writing out original interp path: /lib/ld-linux-aarch64.so.1
Finished.
elfmaster@esoteric-aarch64:~/hello_world$ sudo cp patch.o /opt/shiva/modules/
elfmaster@esoteric-aarch64:~/hello_world$ ./hello
Hello DEFCON 31!
elfmaster@esoteric-aarch64:~/hello_world$ ./hello
Hello DEFCON 31!
elfmaster@esoteric-aarch64:~/hello world$ ./hello
```

Transformations begin where Relocations leave off

- ELF Relocations
  - ELF meta-data that describe simple patching operations
  - Re-encoding instructions and symbolically resolving definitions between code and data
  - No larger than 4 to 8 byte patches
- ELF Transformations
  - ELF meta-data that describe complex patching operations
  - Function Transformation: Granular function re-writing
  - Splice code is fully relocatable with rich symbolic access
  - Designed on top of ELF relocations

Function splicing

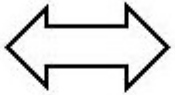
- Splice code into existing function
  - Re-write any portion of a function
  - Relocatable code is spliced into arbitrary locations with the help of *ELF Transformations (Short name: Transforms)*
  - Rich symbolic access to symbols process wide
  - No limit to the amount of code being spliced in, Shiva will extend the size of the function
- Transform macros
  - Help the developer accomplish granular patching controls
  - Generate custom ELF meta-data called Transform records.
  - Designed on top of ELF relocations as a second layer of abstraction to program transformation

Fixing a strcpy vulnerability

```
#define MAX_BUF_LEN 16

void copy_string(char *src)
{
      char buf[MAX_BUF_LEN];

    - strcpy(buf, src);
}
```

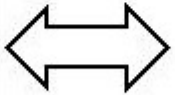


```
#define MAX_BUF_LEN 16

void copy_string(char *src)
{
      char buf[MAX_BUF_LEN];

    + strncpy(buf, src, MAX_BUF_LEN);
    + buf[MAX_BUF_LEN - 1] = '\0';
}
```

```
00000000000007b4 <copy_string>:
 7b4:   stp     x29, x30, [sp, #-48]!
 7b8:   mov     x29, sp
-7bc:   str     x0, [x29, #24]
-7c0:   add     x0, x29, #0x20
-7c4:   ldr     x1, [x29, #24]
-7c8:   bl      690 <strcpy@plt>
 7cc:   nop
 7d0:   ldp     x29, x30, [sp], #48
 7d4:   ret
```



```
00000000000007b4 <copy_string>:
 7b4:   stp     x29, x30, [sp, #-48]!
 7b8:   mov     x29, sp
+7bc:   mov     x9, x29
+7c0:   add     x9, x9, #0x20
+7c4:   mov     x1,x9
+7c8:   mov     x3, x1
+7cc:   mov     x2, #0xf
+7d0:   mov     x1, x0
+7d4:   mov     x0, x3
+7d8:   bl      8d4 <strncpy@patch.plt>
 7dc:   nop
 7e0:   ldp     x29, x30, [sp], #48
 7e4:   ret
```

Patch source to fix vulnerability

```
#include <stdint.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#include "/opt/shiva/include/shiva_module.h"

#define MAX_BUF_LEN 16

SHIVA_T_SPLICE_FUNCTION(copy_string, 0x7bc, 0x7cc)
{
        SHIVA_T_PAIR_X0(src);
        SHIVA_T_LEA_BP_32(dst);
        strncpy(dst, src, MAX_BUF_LEN - 1);
}
```

Patch source to fix vulnerability

```
overflow_patch.o:     file format elf64-littleaarch64


Disassembly of section .text:

0000000000000000 <__shiva_splice_fn_name_copy_string>:
   0:   f81f0ffe        str     x30, [sp, #-16]!
   4:   aa1d03e9        mov     x9, x29
   8:   91006129        add     x9, x9, #0x18
   c:   aa0903e1        mov     x1, x9
  10:   aa0103e3        mov     x3, x1
  14:   d2800202        mov     x2, #0x10
  18:   aa0003e1        mov     x1, x0
  1c:   aa0303e0        mov     x0, x3
  20:   94000000        bl      0 <strncpy>
  24:   d503201f        nop
  28:   f84107fe        ldr     x30, [sp], #16
  2c:   d65f03c0        ret

Disassembly of section .shiva.transform:

0000000000000000 <__shiva_splice_insert_copy_string>:
   0:   000007bc        .word   0x000007bc
   4:   00000000        .word   0x00000000

0000000000000008 <__shiva_splice_extend_copy_string>:
   8:   000007cc        .word   0x000007cc
   c:   00000000        .word   0x00000000
```

Overflow crashes

```
elfmaster@esoteric-aarch64:~/shiva_examples/strcpy_vuln$ ./overflow AAAA
The string: AAAA, was copied into buf!
elfmaster@esoteric-aarch64:~/shiva_examples/strcpy_vuln$ ./overflow AAAAAAAAAAAAAAAAAAAAAAAAAAAA
The string: AAAAAAAAAAAAAAAAAAAAAAAAAAAA, was copied into buf!
Segmentation fault (core dumped)
elfmaster@esoteric-aarch64:~/shiva_examples/strcpy_vuln$ shiva-ld -e overflow -p overflow_patch.o \
> -s /opt/shiva/modules -i /lib/shiva -o overflow
[+] Input executable: overflow
[+] Input search path for patch: /opt/shiva/modules
[+] Basename of patch: overflow_patch.o
[+] Output executable: overflow
Writing out original interp path: /lib/ld-linux-aarch64.so.1
Finished.
elfmaster@esoteric-aarch64:~/shiva_examples/strcpy_vuln$ sudo cp overflow_patch.o /opt/shiva/modules/
elfmaster@esoteric-aarch64:~/shiva_examples/strcpy_vuln$ ./overflow AAAAAAAAAAAAAAAAAAAAAAAAAAAA
The string: AAAAAAAAAAAAAAAAAAAAAAAAAAAA, was copied into buf!
elfmaster@esoteric-aarch64:~/shiva_examples/strcpy_vuln$
```

Spliceable code

- Spliceable code characteristics:
  - Transform data is stored symbolically in .text and .shiva.transform
  - .shiva.transform section contains transform addresses
    - uint64_t ___shiva_splice_insert_##fn_name = <insert_address>
    - uint64_t ___shiva_splice_extend_##fn_name = <extend_address>
  - .text contains splice code i.e.
    - ___shiva_splice_fn_name_##fn_name { ... splice-code ... }
  - Code is fully symbolic and relocatable
  - Rich symbolic access to symbols process wide
  - No limit to size of code being inserted, function is completely re-written
  - Macros for gaining access to live variables on the stack or in registers
    - See available macros in module/include/shiva_module.h

Confluent linking technology

- Cross relocations
  - Shiva and ld-linux.so must synchronize relocation data and work together to solve a relocation
  - In ELF PIE binaries the .bss variables are accessed indirectly through the .got
  - Shiva performs Relocation mangling to instruct RTLD to patch the .got with the offset to the new .bss variable.
- Delayed relocations
  - Relocations in the patch that can't be resolved until the dynamic linker has loaded and linked it's dependencies.
  - This type of relocation is resolved by the Shiva post-linker.
  - Shiva changes AT_ENTRY auxv value to (void *)&shiva_post_linker()

Made open source today

- MIT License
- Shiva v0.11 Alpha. Available today.
- https://github.com/advanced-microcode-patching/shiva

Symbol interposition demo

- ELF aarch64 PIE binary
- Re-define function: `int test_foo(void)`
- Re-declare storage type of: `int lucky_number = 7;`

Function splicing demo

- This patch defines ***splice code*** for target function: int foo(int, char *);
  - Patch adds new logic and functionality to the function
  - Shiva performs much magic under the hood...
- This patch also makes use of symbol interposition

Video editing by Jayden O'Neill

- Thank you to my son Jayden for the wonderful demo video editing!

AKA @gamertries

Questions, contact

- Thank you for attending this talk...
- More about Shiva: https://arcana-research.io/shiva
- Contact: elfmaster@arcana-research.io
- Shiva user manual: https://github.com/advanced-microcode-patching/shiva_user_manual
- Shiva github: https://github.com/advanced-microcode-patching/shiva